Machine Coding: React Toast Notifications

Problem: Build a toast notification system: any component can call `toast.success("Saved")`, toasts stack in a corner, auto-dismiss after a few seconds, pause on hover, and can be closed by hand.
Tech stack: React 19 (Vite, TypeScript)
Time limit: 60 minutes

PART A: Thought process

These steps work for any machine coding task:

1. **Clarify the requirements.** Ask before you build; write the assumptions down.
2. **List the interactions** (what the user does → what they see). This is your checklist.
3. **Sketch the components** and who talks to whom.
4. **Design the state:** the smallest set of values; derive everything else.
5. **Define data shapes and contracts** (the public API other developers call).
6. **Trace one flow end to end** before coding.
7. **Name the hard parts early** so they get time.
8. **Build in layers:** something working on screen first, then add one layer at a time.
9. **Walk through every state** (empty, many, error, edge cases).
10. **Wrap up:** edge cases, trade-offs, what you'd add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, questions, assumptions	5 min
API contract + components + state design	7 min
Core build (provider, context, render list, close button)	15 min
Auto-dismiss timer + pause on hover	10 min
Types/styles, positions, max stack, portal	10 min
Exit animation, accessibility, edge cases	8 min
Wrap-up and walkthrough with the interviewer	5 min

Think out loud. The interviewer grades your reasoning as much as your code. Say what you are about to do and why ("I'll put the API in context so any component can call it without prop drilling"), and say when you take a shortcut on purpose.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A global API: any component can show a toast with one call (`toast.success(msg)`, `toast.error(msg)`, ...).
- Four types: **success, error, info, warning**, each with its own color and icon.
- Toasts **auto-dismiss** after a duration (default 3 s); a duration of `0` means "sticky" (stays until closed).
- Each toast has a **close (×)** button.
- **Hover (or focus) pauses** the timer; leaving resumes it with the time that was left.
- A **progress bar** shows the time left.
- Toasts **stack**; at most `max` (default 3) are visible, the oldest drops off.
- Configurable **position**: top-right, top-left, bottom-right, bottom-left.

Non-functional requirements

- **Performance:** showing a toast must not re-render the components that call `toast.x()`. One timer per visible toast, cleaned up on unmount.
- **Scalability:** many toasts fired quickly (e.g. a loop of failed requests) must not flood the screen: cap the stack.
- **Error handling:** using the hook outside the provider throws a clear error; dismissing an already-removed toast is a no-op.
- **Accessibility:** screen readers announce toasts (`aria-live`), errors use `role="alert"`, the close button has a label and is keyboard reachable, focus pauses the timer, and motion respects `prefers-reduced-motion`.
- **Reusability:** the toast module has no app knowledge; drop `<ToastProvider>` at the root and call `useToast()`.
- **Extensibility:** easy to add a new type, an action button ("Undo"), or a `toast.promise()` helper later.

Clarifying questions to ask the interviewer

1. Is it a global system (call from anywhere) or a single component on one page? → Global.
2. Which types and is the duration configurable per toast? Do we need sticky toasts?
3. Should hover pause the timer? Should there be a progress bar?
4. How many toasts can show at once? What happens to extra ones: drop the oldest or queue them?
5. One fixed position or configurable? Mobile layout?
6. Can I use a library (react-hot-toast, sonner)? → No, build it from scratch.

Assumptions we will build with

- Global system through **Context + a provider** at the app root; consumers use a `useToast()` hook.
- Types: success, error, info, warning. Default duration **3000 ms**, per-toast override, `0` = sticky.
- Hover or keyboard focus pauses the timer and the progress bar.
- **Max 3** visible; a new toast pushes out the **oldest** (no hidden queue).

- One position for the whole provider (prop), default top-right. New toasts appear at the bottom of the stack for top positions and at the top for bottom positions (closest to the screen edge = oldest).
- No external libraries. Plain CSS.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Call `toast.success("Profile saved")`	Green toast with ✓ slides in at the chosen corner
Call `toast.error("Payment failed")`	Red toast with ✕, announced immediately (`role="alert"`)
Wait 3 s	Toast animates out and is removed
Hover the toast	Timer and progress bar pause
Mouse leaves after 1 s of hover	Timer resumes with the time that was left (not a full reset)
Tab to the close button	Timer pauses (focus), button shows focus ring
Click × (or Enter on it)	Toast animates out and is removed
Call with `{ duration: 0 }`	Toast stays, no progress bar, only × closes it
Fire a 4th toast while 3 are visible	Oldest toast is removed, new one is added
Change the position prop	Container moves to the new corner, toasts stay
Async API call fails	Caller shows `toast.error(...)` in its `catch`
`useToast()` outside the provider	Throws "useToast must be used inside <ToastProvider>"
User has reduced motion turned on	No slide animations, toasts still appear and disappear

This table is the implementation and test checklist.

---

3. Design the architecture

Module	Responsibility
`types.ts`	Shapes: `Toast`, `ToastType`, `ToastOptions`, `ToastApi`
`toastReducer.ts`	Pure list logic: add (with max), mark leaving, remove
`ToastContext.ts`	The context object and the `useToast()` hook
`ToastProvider.tsx`	Owns the toast list, builds the stable API, renders the container in a portal
`ToastItem.tsx`	One toast: icon, message, close button, its own timer + pause, progress bar
`toast.css`	Positions, colors per type, enter/exit/progress animations
`App.tsx`	Demo: buttons that fire each type, position picker, mock API call

```
<App>
└── <ToastProvider position max duration>     owns toasts[], provides api
    ├── {children}                            any component → useToast() → api.success(...)
    │   └── <Demo/>
    └── createPortal(document.body)
        └── <div.toast-container aria-live>
            └── <ToastItem/> × n              each owns its own timer
```

Start simple: provider + list + close button first. The timer lives in `ToastItem` because each toast pauses independently; the list logic lives in a pure reducer because it is easy to test.

---

4. Design the state / data model

State or field	Purpose
`toasts: Toast[]` (provider)	The visible toasts, in insertion order
`toast.id`	Stable key for React and for `dismiss(id)`
`toast.type`, `toast.message`	What to show
`toast.duration`	Auto-dismiss time in ms, `0` = sticky
`toast.leaving`	True while the exit animation plays, before removal
`paused` (ToastItem state)	Hover/focus pauses the timer and progress bar
`remaining` (ToastItem ref)	Time left, survives pause/resume without causing renders

```
const [toasts, dispatch] = useReducer(toastReducer, []);   // ToastProvider

const [paused, setPaused] = useState(false);               // ToastItem
const remaining = useRef(duration);
```

Senior note:

- **Derived, not stored:** the visible count (`toasts.length`), the icon and color (from `type`), and whether to show a progress bar (`duration > 0`) are all computed while rendering.
- **Ownership:** the provider owns *which* toasts exist. Each `ToastItem` owns *its own* timer and pause state, because that state matters to nobody else. `remaining` is a ref, not state: changing it must not re-render.
- **Separate the API from the list:** the context value is only the API (functions). The list never goes into context, so callers do not re-render when toasts change.

---

5. Design the data structures and contracts

Example data

```
const toasts: Toast[] = [
  { id: '6f1c…', type: 'success', message: 'Profile saved', duration: 3000, leaving: false },
  { id: '9a42…', type: 'error', message: 'Payment failed. Try again.', duration: 3000, leaving: false },
  { id: 'c7d0…', type: 'info', message: 'I stay until you close me', duration: 0, leaving: false },
];
```

Ids come from `crypto.randomUUID()`: stable keys mean React keeps each toast's DOM (and its running timer and animation) when another toast is added or removed. Never use the array index as the key: removing the first toast would shift every key and restart every timer.

Public API contract (what other developers call)

```
const toast = useToast();

toast.show(message: string, options?: { type?: ToastType; duration?: number }): string; // returns id
toast.success(message, { duration? }): string;
toast.error(message, { duration? }): string;
toast.info(message, { duration? }): string;
toast.warning(message, { duration? }): string;
toast.dismiss(id: string): void;

<ToastProvider position="top-right" max={3} duration={3000}>…</ToastProvider>
```

Example:

```
const id = toast.info('Uploading…', { duration: 0 });
await upload();
toast.dismiss(id);
toast.success('Upload complete');
```

Mock API used in the demo

```
// fails about half the time so both toasts can be seen
const fakeSave = () =>
  new Promise<void>((resolve, reject) => setTimeout(() => (Math.random() > 0.5 ? resolve() : reject()),
600));
```

---

6. Trace the data flow

"User clicks Save, the API fails, an error toast appears and leaves"

```
click "Call API"
   ↓
save(): await fakeSave()  → rejects after 600 ms
   ↓
catch → toast.error('Could not save profile. Try again.')
   ↓
api.show(): id = randomUUID(); dispatch({ type: 'ADD', toast, max: 3 })
   ↓
toastReducer: [...state, toast].slice(-3)        (oldest drops if 4)
   ↓
ToastProvider re-renders → portal renders <ToastItem key={id}/>
   ↓
ToastItem mounts → enter animation, role="alert" → screen reader announces
   ↓
useEffect: setTimeout(onDismiss(id), remaining = 3000)
   ↓   (user hovers at 1 s → cleanup clears timer, remaining = 2000; leaves → new 2000 ms timer)
timer fires → api.dismiss(id)
   ↓
dispatch LEAVE → toast.leaving = true → CSS exit animation (200 ms)
   ↓
setTimeout 200 ms → dispatch REMOVE → toast unmounts
```

---

7. Important technical challenges

Challenge 1: A global API without re-rendering every caller

Problem: A form component calls `toast.success()`. If the context value were `{ toasts, addToast }`, every component using `useToast()` would re-render each time any toast is added or removed, even though it only needs the function. With 50 consumers and a burst of toasts, that is a lot of wasted renders.

Solution: Put only the API in context and make it stable with `useMemo` (it only changes when `max` or `duration` props change). `dispatch` from `useReducer` is already stable. The toast list stays local to the provider.

```
const [toasts, dispatch] = useReducer(toastReducer, []);
const api = useMemo<ToastApi>(() => {
  const show = (message: string, options: ToastOptions = {}) => {
    const id = crypto.randomUUID();
    dispatch({ type: 'ADD', max, toast: { id, message, type: options.type ?? 'info',
      duration: options.duration ?? duration, leaving: false } });
    return id;
  };
  // ...dismiss, success, error, info, warning
}, [max, duration]);

return <ToastContext value={api}>{children}{createPortal(list, document.body)}</ToastContext>;
```

Challenge 2: Auto-dismiss that pauses and resumes

Problem: A 3 s toast is hovered at 1 s for 5 s. A naive `setTimeout(3000)` on mount removes it while the user is reading. Restarting a fresh 3 s timer on mouse leave is also wrong: the toast now lives 4 s in total of visible time.

Solution: Keep the time left in a ref. The effect starts a timer for `remaining`; when it is cleaned up (pause, leaving, unmount) it subtracts the elapsed time. Hover/focus toggles `paused`, which re-runs the effect. The CSS progress bar uses the same `paused` flag through `animation-play-state`, so the bar and the timer stay in sync.

```
const [paused, setPaused] = useState(false);
const remaining = useRef(duration);

useEffect(() => {
  if (paused || leaving || duration === 0) return;
  const start = Date.now();
  const timer = setTimeout(() => onDismiss(id), remaining.current);
  return () => {
    clearTimeout(timer);
    remaining.current -= Date.now() - start;
  };
}, [paused, leaving, duration, id, onDismiss]);
```

Challenge 3: Exit animation before removal

Problem: If `dismiss` removes the toast from the list right away, React unmounts it in the same frame and there is no time to animate it out: the toast just vanishes and the others jump.

Solution: Two steps. `dismiss` marks the toast `leaving` (CSS plays `toast-out`), then removes it after the animation length. Use a timer, not `onAnimationEnd`: with `prefers-reduced-motion` the animation is off, `animationend` never fires, and the toast would stay forever.

```
const dismiss = (id: string) => {
  dispatch({ type: 'LEAVE', id });
  setTimeout(() => dispatch({ type: 'REMOVE', id }), EXIT_MS); // EXIT_MS = 200, same as CSS
};
```

Challenge 4: Stack limit, layering and accessibility

Problem: A loop of 20 failed requests fires 20 toasts and covers the page. The container sits inside an `overflow: hidden` card and gets clipped. A screen reader user never hears that their payment failed.

Solution:

- The reducer keeps only the newest `max` toasts: `[...state, toast].slice(-max)`. Dropped toasts unmount, and their effect cleanup clears their timers.
- Render the container through `createPortal(…, document.body)` with `position: fixed` and a high `z-index`, so no parent can clip it. `pointer-events: none` on the container, `auto` on toasts, so the empty area never blocks clicks.
- The container is an always-mounted `aria-live="polite"` region (live regions must exist *before* content is added). Error toasts add `role="alert"` (assertive). The close button has `aria-label`, focus pauses the timer.

```
case 'ADD':
  return [...state, action.toast].slice(-action.max);
```

---

8. Implement in the correct order

#	Step	Working result
1	`types.ts`: `Toast`, `ToastType`, `ToastApi`	Shapes agreed
2	`toastReducer` with ADD / REMOVE + a quick assert	List logic tested
3	`ToastContext` + `useToast()` with the "outside provider" error	Hook ready
4	`ToastProvider` with `useReducer` and a `show` function; render messages as plain text	Clicking a button shows text
5	`ToastItem` with type class, icon and close button → `dismiss`	Toasts look right, can be closed
6	Auto-dismiss timer in `ToastItem`	Toasts disappear after 3 s
7	Pause on hover/focus with `remaining` ref	Hover keeps the toast
8	Progress bar with `animation-play-state`	Time left visible, pauses with hover
9	Portal + fixed container + positions + `max`	Corners work, stack is capped
10	`LEAVE` action + exit animation + delayed `REMOVE`	Smooth exit
11	ARIA (`aria-live`, `role="alert"`, labels), reduced motion	Accessible
12	Demo page: all types, sticky, mock API call, position select	Everything shown end to end

9. Cover all states and edge cases

State / edge case	How it is handled
Initial (no toasts)	Empty container stays mounted (live region ready), takes no clicks
One toast	Shows, progress bar runs, auto-dismisses
Many toasts quickly	Reducer keeps the newest `max`; dropped toasts' timers are cleared on unmount
Hovered toast	Timer and progress bar paused
Hover → leave → hover again	`remaining` keeps shrinking correctly across pauses
Focus via keyboard	Same as hover: paused while focused
Sticky (`duration: 0`)	No timer, no progress bar, only × closes it
Close clicked while timer about to fire	`LEAVE` and `REMOVE` are idempotent; `leaving` stops the timer effect
`dismiss(id)` for an unknown/removed id	No-op (map/filter find nothing)
Long message	`overflow-wrap: anywhere` , toast grows in height
Narrow screen	Container width `min(360px, 100vw - 32px)`
Position changed while toasts are visible	Only the container class changes; toasts keep their timers
Error from async call	Caller shows `toast.error` in `catch`
Hook used outside provider	Clear error thrown
Reduced motion	Animations off; removal still happens by timer
React StrictMode double effects	Effect cleanup subtracts ~0 ms and restarts; behavior unchanged

10. Final structure

```
_25_react_toast/
├── index.html
├── package.json
└── src/
    ├── main.tsx
    ├── index.css            page base styles
    ├── App.tsx              demo: buttons, position select, mock API
    ├── App.css
    └── toast/
        ├── types.ts
        ├── toastReducer.ts  pure list logic (tested)
```

```
        ├── ToastContext.ts  context + useToast()
        ├── ToastProvider.tsx
        ├── ToastItem.tsx
        └── toast.css
```

---

11. Full working code

`src/toast/types.ts`

```ts
export type ToastType = 'success' | 'error' | 'info' | 'warning';

export type ToastPosition = 'top-right' | 'top-left' | 'bottom-right' | 'bottom-left';

export interface Toast {
  id: string;
  type: ToastType;
  message: string;
  duration: number; // ms, 0 = stays until the user closes it
  leaving: boolean; // true while the exit animation plays
}

export interface ToastOptions {
  type?: ToastType;
  duration?: number;
}

type Shortcut = (message: string, options?: Omit<ToastOptions, 'type'>) => string;

export interface ToastApi {
  show: (message: string, options?: ToastOptions) => string;
  success: Shortcut;
  error: Shortcut;
  info: Shortcut;
  warning: Shortcut;
  dismiss: (id: string) => void;
}
```

`src/toast/toastReducer.ts`

```ts
import type { Toast } from './types';

export type ToastAction =
  | { type: 'ADD'; toast: Toast; max: number }
  | { type: 'LEAVE'; id: string }
```

```
  | { type: 'REMOVE'; id: string };

export function toastReducer(state: Toast[], action: ToastAction): Toast[] {
  switch (action.type) {
    case 'ADD':
      // keep only the newest `max` toasts: the oldest drop off
      return [...state, action.toast].slice(-action.max);
    case 'LEAVE':
      return state.map((t) => (t.id === action.id ? { ...t, leaving: true } : t));
    case 'REMOVE':
      return state.filter((t) => t.id !== action.id);
  }
}
```

`src/toast/ToastContext.ts`

```
import { createContext, useContext } from 'react';
import type { ToastApi } from './types';

export const ToastContext = createContext<ToastApi | null>(null);

export function useToast(): ToastApi {
  const api = useContext(ToastContext);
  if (!api) throw new Error('useToast must be used inside <ToastProvider>');
  return api;
}
```

`src/toast/ToastProvider.tsx`

```
import { useMemo, useReducer, type ReactNode } from 'react';
import { createPortal } from 'react-dom';
import { ToastContext } from './ToastContext';
import { ToastItem } from './ToastItem';
import { toastReducer } from './toastReducer';
import type { ToastApi, ToastOptions, ToastPosition } from './types';
import './toast.css';

const EXIT_MS = 200; // same length as the toast-out animation in toast.css

interface Props {
  children: ReactNode;
  position?: ToastPosition;
  max?: number;
  duration?: number;
}
```

```
export function ToastProvider({ children, position = 'top-right', max = 3, duration = 3000 }: Props) {
  const [toasts, dispatch] = useReducer(toastReducer, []);

  // stable api object: components that only call toast.x() never re-render on new toasts
  const api = useMemo<ToastApi>(() => {
    const show = (message: string, options: ToastOptions = {}) => {
      const id = crypto.randomUUID();
      dispatch({
        type: 'ADD',
        max,
        toast: { id, message, type: options.type ?? 'info', duration: options.duration ?? duration,
leaving: false },
      });
      return id;
    };
    const dismiss = (id: string) => {
      dispatch({ type: 'LEAVE', id });
      // a timer, not onAnimationEnd: it still fires when animations are off (reduced motion)
      setTimeout(() => dispatch({ type: 'REMOVE', id }), EXIT_MS);
    };
    return {
      show,
      dismiss,
      success: (message, options) => show(message, { ...options, type: 'success' }),
      error: (message, options) => show(message, { ...options, type: 'error' }),
      info: (message, options) => show(message, { ...options, type: 'info' }),
      warning: (message, options) => show(message, { ...options, type: 'warning' }),
    };
  }, [max, duration]);

  return (
    <ToastContext value={api}>
      {children}
      {createPortal(
        <div className={`toast-container ${position}`} role="region" aria-label="Notifications" aria-
live="polite">
          {toasts.map((t) => (
            <ToastItem key={t.id} toast={t} onDismiss={api.dismiss} />
          ))}
        </div>,
        document.body,
      )}
    </ToastContext>
  );
}
```

`src/toast/ToastItem.tsx`

```
import { useEffect, useRef, useState } from 'react';
import type { Toast, ToastType } from './types';

const ICONS: Record<ToastType, string> = { success: '✓', error: '✕', info: 'i', warning: '!' };

interface Props {
  toast: Toast;
  onDismiss: (id: string) => void;
}

export function ToastItem({ toast, onDismiss }: Props) {
  const { id, type, message, duration, leaving } = toast;
  const [paused, setPaused] = useState(false);
  const remaining = useRef(duration);

  // auto-dismiss timer that pauses on hover/focus and resumes with the time left
  useEffect(() => {
    if (paused || leaving || duration === 0) return;
    const start = Date.now();
    const timer = setTimeout(() => onDismiss(id), remaining.current);
    return () => {
      clearTimeout(timer);
      remaining.current -= Date.now() - start;
    };
  }, [paused, leaving, duration, id, onDismiss]);

  return (
    <div
      className={`toast ${type}${leaving ? ' leaving' : ''}`}
      role={type === 'error' ? 'alert' : undefined}
      onMouseEnter={() => setPaused(true)}
      onMouseLeave={() => setPaused(false)}
      onFocus={() => setPaused(true)}
      onBlur={() => setPaused(false)}
    >
      <span className="toast-icon" aria-hidden="true">
        {ICONS[type]}
      </span>
      <p className="toast-message">{message}</p>
      <button type="button" className="toast-close" aria-label="Dismiss notification" onClick={() =>
onDismiss(id)}>
        ×
      </button>
      {duration > 0 && (
        <span
          className="toast-progress"
```

```
          style={{ animationDuration: `${duration}ms`, animationPlayState: paused ? 'paused' : 'running'
}}
        />
      )}
    </div>
  );
}
```

`src/toast/toast.css`

```
.toast-container {
  position: fixed;
  z-index: 1000;
  display: flex;
  flex-direction: column;
  gap: 10px;
  width: min(360px, calc(100vw - 32px));
  pointer-events: none; /* the empty area never blocks clicks on the page */
}
.toast-container.top-right { top: 16px; right: 16px; }
.toast-container.top-left { top: 16px; left: 16px; }
.toast-container.bottom-right { bottom: 16px; right: 16px; flex-direction: column-reverse; }
.toast-container.bottom-left { bottom: 16px; left: 16px; flex-direction: column-reverse; }

.toast {
  pointer-events: auto;
  position: relative;
  overflow: hidden;
  display: flex;
  align-items: flex-start;
  gap: 10px;
  padding: 12px 14px;
  border-radius: 8px;
  border-left: 4px solid var(--accent);
  background: #fff;
  color: #1f2937;
  box-shadow: 0 6px 20px rgb(0 0 0 / 0.15);
  animation: toast-in 200ms ease-out;
}
.toast.leaving { animation: toast-out 200ms ease-in forwards; }

.toast.success { --accent: #16a34a; }
.toast.error { --accent: #dc2626; }
.toast.info { --accent: #2563eb; }
.toast.warning { --accent: #d97706; }

.toast-icon {
```

```
  flex: none;
  display: grid;
  place-items: center;
  width: 22px;
  height: 22px;
  border-radius: 50%;
  background: var(--accent);
  color: #fff;
  font-size: 13px;
  font-weight: 700;
}
.toast-message { flex: 1; margin: 0; line-height: 1.4; overflow-wrap: anywhere; }
.toast-close {
  flex: none;
  border: 0;
  background: none;
  padding: 0 4px;
  font-size: 20px;
  line-height: 1;
  color: #6b7280;
  cursor: pointer;
}
.toast-close:focus-visible { outline: 2px solid var(--accent); border-radius: 4px; }

.toast-progress {
  position: absolute;
  left: 0;
  bottom: 0;
  width: 100%;
  height: 3px;
  background: var(--accent);
  transform-origin: left;
  animation-name: toast-progress;
  animation-timing-function: linear;
  animation-fill-mode: forwards;
}

@keyframes toast-in { from { opacity: 0; transform: translateY(-8px); } }
@keyframes toast-out { to { opacity: 0; transform: translateX(110%); } }
@keyframes toast-progress { to { transform: scaleX(0); } }

@media (prefers-reduced-motion: reduce) {
  .toast, .toast.leaving { animation: none; }
}
```

`src/App.tsx`

```
import { useState } from 'react';
import { useToast } from './toast/ToastContext';
import { ToastProvider } from './toast/ToastProvider';
import type { ToastPosition } from './toast/types';
import './App.css';

const POSITIONS: ToastPosition[] = ['top-right', 'top-left', 'bottom-right', 'bottom-left'];

// mock API: fails about half the time
const fakeSave = () =>
  new Promise<void>((resolve, reject) => setTimeout(() => (Math.random() > 0.5 ? resolve() : reject()),
600));

function Demo() {
  const toast = useToast();

  const save = async () => {
    try {
      await fakeSave();
      toast.success('Profile saved');
    } catch {
      toast.error('Could not save profile. Try again.');
    }
  };

  return (
    <div className="buttons">
      <button onClick={() => toast.success('Profile saved')}>Success</button>
      <button onClick={() => toast.error('Payment failed. Try again.')}>Error</button>
      <button onClick={() => toast.info('A new version is available')}>Info</button>
      <button onClick={() => toast.warning('Storage almost full', { duration: 6000 })}>Warning (6s)
</button>
      <button onClick={() => toast.info('I stay until you close me', { duration: 0 })}>Sticky</button>
      <button onClick={save}>Call API</button>
    </div>
  );
}

export default function App() {
  const [position, setPosition] = useState<ToastPosition>('top-right');

  return (
    <ToastProvider position={position}>
      <main className="app">
        <h1>React Toast</h1>
        <label>
```

```
          Position{' '}
          <select value={position} onChange={(e) => setPosition(e.target.value as ToastPosition)}>
            {POSITIONS.map((p) => (
              <option key={p}>{p}</option>
            ))}
          </select>
        </label>
        <Demo />
      </main>
    </ToastProvider>
  );
}
```

`src/App.css`

```
.app {
  max-width: 640px;
  margin: 0 auto;
  padding: 48px 16px;
}
.app label { display: block; margin-bottom: 20px; }
.app select { padding: 6px 8px; font: inherit; }
.buttons { display: flex; flex-wrap: wrap; gap: 10px; }
.buttons button {
  padding: 10px 16px;
  border: 1px solid #d1d5db;
  border-radius: 6px;
  background: #fff;
  font: inherit;
  cursor: pointer;
}
.buttons button:hover { background: #f3f4f6; }
```

`src/index.css`

```
body {
  margin: 0;
  font-family: system-ui, sans-serif;
  background: #f3f4f6;
  color: #111827;
}
```

`src/main.tsx` stays as the Vite template (`<StrictMode><App /></StrictMode>`).

Quick self-check of the reducer (`node check.ts`)

```
import assert from 'node:assert/strict';
import { toastReducer } from './app/src/toast/toastReducer.ts';

const make = (id: string) => ({ id, type: 'info' as const, message: id, duration: 3000, leaving: false
});

let s = toastReducer([], { type: 'ADD', toast: make('a'), max: 3 });
s = toastReducer(s, { type: 'ADD', toast: make('b'), max: 3 });
s = toastReducer(s, { type: 'ADD', toast: make('c'), max: 3 });
s = toastReducer(s, { type: 'ADD', toast: make('d'), max: 3 });
assert.deepEqual(s.map((t) => t.id), ['b', 'c', 'd'], 'oldest drops when over max');

s = toastReducer(s, { type: 'LEAVE', id: 'c' });
assert.equal(s.find((t) => t.id === 'c')?.leaving, true, 'leave marks the toast');
assert.equal(s.length, 3, 'leave keeps it in the list for the exit animation');

s = toastReducer(s, { type: 'REMOVE', id: 'c' });
s = toastReducer(s, { type: 'REMOVE', id: 'c' });
assert.deepEqual(s.map((t) => t.id), ['b', 'd'], 'remove is idempotent');

console.log('reducer checks passed');
```

---

12. Wrap-up

Edge cases handled

- Burst of toasts capped at `max`, oldest drops, timers cleaned up.
- Pause/resume keeps the real time left; progress bar stays in sync.
- Sticky toasts, manual close, double dismiss, unknown id.
- Exit animation that still removes the toast when animations are disabled.
- Portal + fixed position so no parent clips the toasts; empty container never blocks clicks.
- Screen reader announcements, `role="alert"` for errors, keyboard focus pauses.

Trade-offs to mention

- **Context vs a global store/event emitter:** Context is built in and enough here. The cost: `toast()` can only be called from React components (inside the provider). A tiny external store (`subscribe` + `useSyncExternalStore`) would allow calling it from plain functions like an axios interceptor.
- **Drop oldest vs queue:** dropping is simpler and keeps messages fresh; a queue never loses a message but can show stale ones late.

- **Timer per toast vs one global ticker:** per-toast timers are simple and pause independently; a single `requestAnimationFrame` loop would scale better for hundreds of toasts (not a real case for toasts).
- **CSS progress bar vs JS state:** CSS animation costs zero renders; the timer and the bar are two clocks kept in sync by the same `paused` flag, which can drift by a few ms.
- **Fixed 200 ms exit timer** must match the CSS duration; a CSS variable or `onAnimationEnd` with fallback would remove the duplicate number.

What to add with more time

- **Tests:** React Testing Library + fake timers: shows on call, removes after duration, pauses on hover, caps at max, `useToast` throws outside provider.
- `toast.promise(promise, { loading, success, error })` that updates one toast in place (`UPDATE` action).
- Action buttons ("Undo"), custom JSX content, per-toast position.
- Dedupe identical messages (count badge "×3" instead of 3 toasts).
- Swipe to dismiss on mobile, `Escape` to close the latest, pause all when the tab is hidden (`visibilitychange`).
- Call from outside React via an external store.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why build it with Context instead of passing an `onToast` prop down?

Toasts are triggered from deep, unrelated places (forms, API layers, modals). Prop drilling would touch every component in between. Context gives one provider at the root and a `useToast()` hook anywhere below it.

Q2. What if the product wants to call toast from a non-React file, like an axios interceptor?

Hooks only work inside components. Move the list into a small external store (a module with `toasts` , `listeners` , `toast()` and `subscribe()`), and let the provider read it with `useSyncExternalStore` . Then `import { toast } from './toast'` works anywhere. This is how react-hot-toast and sonner work.

Q3. What if the interviewer says "never lose a message" instead of dropping the oldest?

Keep two lists: `visible` (max 3) and `queue` . On add, push to the queue when visible is full; on remove, move the next one from the queue to visible. Mention that a long queue may show stale messages, so you might expire queued items too.

State design

Q4. Why is `remaining` a ref and not state?

It changes on every pause, but the UI never shows it (the CSS bar shows progress). State would cause a re-render for nothing; a ref keeps the value across renders silently.

Q5. Why does each `ToastItem` own its timer instead of the provider?

Each toast pauses on its own hover. Keeping the timer next to the hover state means no maps of timer ids in the provider, and React cleans the timer up automatically when the toast unmounts.

Q6. Why `useReducer` and not `useState`?

Three transitions (add with cap, mark leaving, remove) are easier to read and test as a pure function. `dispatch` is also stable, so the memoized API never needs to change because of it.

Q7. Why not use the array index as the key?

Removing the first toast shifts all indexes, so React reuses the wrong DOM nodes: animations replay and the wrong timer state stays with the wrong message. A stable id keeps each toast's component (and its timer) attached to its data.

Challenge 1: re-renders

Q8. How do you prove callers don't re-render when a toast is added?

Put a `console.count` or React DevTools "highlight updates" on the `Demo` component and fire toasts. Only the provider's subtree in the portal re-renders. The context value is a memoized object that does not change, so consumers are not notified.

Q9. The provider re-renders `{children}` on every toast. Isn't that the whole app?

No. `children` is a prop created by the parent (`App`). When only the provider's own state changes, the `children` elements are the same objects, so React skips them. Only the portal list updates.

Challenge 2: timer

Q10. What happens if the user hovers in and out very fast, many times?

Each pause runs the cleanup, which subtracts the real elapsed time and clears the timer; each resume starts one new timer for what is left. There is never more than one timer per toast, and the total visible time stays correct.

Q11. What if the browser tab is in the background?

Browsers throttle timers in background tabs, and the user didn't see the toast anyway. Improvement: listen to `visibilitychange` and set `paused` while `document.hidden`, so toasts wait until the user comes back.

Q12. What about React StrictMode running effects twice?

In development, React mounts, cleans up and mounts again. Our cleanup subtracts the few ms that passed and clears the timer, then a new timer starts. The result is the same, which is exactly what StrictMode checks.

Challenge 3: exit animation

Q13. Why a `setTimeout` for removal and not `onAnimationEnd`?

`animationend` never fires when the animation is disabled (reduced motion, hidden tab in some browsers, a CSS override). A timer always fires. The cost is that `EXIT_MS` must match the CSS duration; `onAnimationEnd` plus a fallback timer would be the most robust version.

Q14. Why not use a library like Framer Motion for the exit?

`AnimatePresence` solves exactly this, but it's a dependency for one 200 ms fade. A `leaving` flag plus CSS keyframes is about 10 lines and has no bundle cost. With more complex layout animations (other toasts sliding up smoothly) a library becomes worth it.

Challenge 4: stack, layering, accessibility

Q15. Why render with a portal?

A toast container inside a parent with `overflow: hidden`, `transform`, or a lower stacking context can be clipped or hidden behind other content. Portaling to `document.body` with `position: fixed` puts it above the app regardless of where the provider sits.

Q16. Why is the container always mounted, even when empty?

Screen readers only watch `aria-live` regions that already exist. If the region is added together with the first toast, many screen readers miss the announcement.

Q17. Why `role="alert"` only for errors?

`alert` is assertive: it interrupts what the screen reader is saying. That is right for "Payment failed" but annoying for "Saved". Normal toasts use the polite region and are read when the reader is idle.

Q18. A toast disappears in 3 s. Is that accessible?

WCAG 2.2.1 says users must be able to extend time limits. Pausing on hover and focus helps, and errors should get longer durations or be sticky. With more time: a global setting for slower toasts, and never put the only copy of important info (or an "Undo" action) in a toast that auto-hides.

Implementation steps

Q19. How would you add an "Undo" button?

Add `action?: { label: string; onClick: () => void }` to `ToastOptions` and the `Toast` shape, render a button next to the message, and call `action.onClick()` then `dismiss(id)` on click. Give such toasts a longer default duration.

Q20. How would you implement `toast.promise()`?

Show a sticky loading toast, keep its id, then on resolve/reject dispatch an `UPDATE` action that changes `type`, `message` and `duration` in place (same id, so the same DOM node updates instead of a new toast popping in).

```
const id = show(msgs.loading, { duration: 0 });
promise.then(() => update(id, { type: 'success', message: msgs.success, duration: 3000 }),
             () => update(id, { type: 'error', message: msgs.error, duration: 3000 }));
```

Q21. How would you stop the same error from appearing 5 times?

In `ADD`, check if a non-leaving toast with the same `type + message` exists. If yes, don't add a new one; instead bump a `count` field and reset its timer (show "×5"). Or let callers pass an explicit `id` and treat a repeated id as an update.

Performance and scale

Q22. What if 1,000 toasts are fired in one second?

The reducer caps the list, so at most `max` toasts render and at most `max` timers run. Each `ADD` is one cheap array copy of ≤ 4 items. React batches the dispatches from one event into one render. For truly huge bursts, dedupe or throttle at the call site.

Q23. Is `[...state, toast].slice(-max)` a problem for performance?

No. The array never holds more than `max + 1` items, so the copy is constant time. It also returns a new array, which React needs to detect the change.

Trade-offs

Q24. Why not just use react-hot-toast or sonner?

In a real product I would, they solve queueing, swipe, promise toasts and edge cases well. In the interview the goal is to show the design underneath: a global API, timers, cleanup, animation and accessibility, which is exactly what those libraries do internally.

Q25. How would you test this?

Unit-test the reducer (done with node asserts). With React Testing Library and `vi.useFakeTimers()`: render the provider with a button, click it, assert the text is there, `advanceTimersByTime(3000 + 200)`, assert it is gone; fire `mouseEnter`, advance time, assert it stays; fire 4 toasts, assert only the 3 newest are shown; render `useToast` outside the provider and expect a throw.